JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface NavigableMap<K,V>

Type Parameters:

`K` - the type of keys maintained by this map

`V` - the type of mapped values

All Superinterfaces:

`Map<K,V>`, `SequencedMap<K,V>`, `SortedMap<K,V>`

All Known Subinterfaces:

`ConcurrentNavigableMap<K,V>`

All Known Implementing Classes:

`ConcurrentSkipListMap`, `TreeMap`

```
public interface NavigableMap<K,V>
extends SortedMap<K,V>
```

A `SortedMap` extended with navigation methods returning the closest matches for given search targets. Methods `lowerEntry(K)`, `floorEntry(K)`, `ceilingEntry(K)`, and `higherEntry(K)` return `Map.Entry` objects associated with keys respectively less than, less than or equal, greater than or equal, and greater than a given key, returning `null` if there is no such key. Similarly, methods `lowerKey(K)`, `floorKey(K)`, `ceilingKey(K)`, and `higherKey(K)` return only the associated keys. All of these methods are designed for locating, not traversing entries.

A `NavigableMap` may be accessed and traversed in either ascending or descending key order. The `descendingMap()` method returns a view of the map with the senses of all relational and directional methods inverted. The performance of ascending operations and views is likely to be faster than that of descending ones. Methods `subMap(K, boolean, K, boolean)`, `headMap(K, boolean)`, and `tailMap(K, boolean)` differ from the like-named `SortedMap` methods in accepting additional arguments describing whether lower and upper bounds are inclusive versus exclusive. Submaps of any `NavigableMap` must implement the `NavigableMap` interface.

This interface additionally defines methods `firstEntry()`, `pollFirstEntry()`, `lastEntry()`, and `pollLastEntry()` that return and/or remove the least and greatest mappings, if any exist, else returning `null`.

The methods `ceilingEntry(K)`, `firstEntry()`, `floorEntry(K)`, `higherEntry(K)`, `lastEntry()`, `lowerEntry(K)`, `pollFirstEntry()`, and `pollLastEntry()` return `Map.Entry` instances that represent snapshots of mappings as of the time of the call. They do *not* support mutation of the underlying map via the optional `setValue` method.

Methods `subMap(K, K)`, `headMap(K)`, and `tailMap(K)` are specified to return `SortedMap` to allow existing implementations of `SortedMap` to be compatibly retrofitted to implement `NavigableMap`, but extensions and implementations of this interface are encouraged to override these methods to return `NavigableMap`. Similarly, `SortedMap.keySet()` can be overridden to return `NavigableSet`.

This interface is a member of the Java Collections Framework.

Since:

1.6

Nested Class Summary

Nested classes/interfaces declared in interface Map
`Map.Entry<K,V>`

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
`Map.Entry<K,V>`	**`ceilingEntry`**`(K key)`	Returns a key-value mapping associated with the least key greater than or equal to the given key, or `null` if there is no such key.
`K`	**`ceilingKey`**`(K key)`	Returns the least key greater than or equal to the given key, or `null` if there is no such key.
`NavigableSet<K>`	**`descendingKeySet`**`()`	Returns a reverse order `NavigableSet` view of the keys contained in this map.
`NavigableMap<K,V>`	**`descendingMap`**`()`	Returns a reverse order view of the mappings contained in this map.
`Map.Entry<K,V>`	**`firstEntry`**`()`	Returns a key-value mapping associated with

		the least key in this map, or `null` if the map is empty.
`Map.Entry<K,V>`	**`floorEntry`**`(K key)`	Returns a key-value mapping associated with the greatest key less than or equal to the given key, or `null` if there is no such key.
`K`	**`floorKey`**`(K key)`	Returns the greatest key less than or equal to the given key, or `null` if there is no such key.
`SortedMap<K,V>`	**`headMap`**`(K toKey)`	Returns a view of the portion of this map whose keys are strictly less than `toKey`.
`NavigableMap<K,V>`	**`headMap`**`(K toKey, boolean inclusive)`	Returns a view of the portion of this map whose keys are less than (or equal to, if `inclusive` is true) `toKey`.
`Map.Entry<K,V>`	**`higherEntry`**`(K key)`	Returns a key-value mapping associated with the least key strictly greater than the given key, or `null` if there is no such key.
`K`	**`higherKey`**`(K key)`	Returns the least key strictly greater than the given key, or `null` if there is no such key.
`Map.Entry<K,V>`	**`lastEntry`**`()`	Returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty.

Map.Entry<K,V>	**lowerEntry**(K key)	Returns a key-value mapping associated with the greatest key strictly less than the given key, or `null` if there is no such key.
K	**lowerKey**(K key)	Returns the greatest key strictly less than the given key, or `null` if there is no such key.
NavigableSet<K>	**navigableKeySet**()	Returns a NavigableSet view of the keys contained in this map.
Map.Entry<K,V>	**pollFirstEntry**()	Removes and returns a key-value mapping associated with the least key in this map, or `null` if the map is empty (optional operation).
Map.Entry<K,V>	**pollLastEntry**()	Removes and returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty (optional operation).
default NavigableMap<K,V>	**reversed**()	Returns a reverse-ordered view of this map.
NavigableMap<K,V>	**subMap**(K fromKey, boolean fromInclusive, K toKey, boolean toInclusive)	Returns a view of the portion of this map whose keys range from `fromKey` to `toKey`.
SortedMap<K,V>	**subMap**(K fromKey, K toKey)	Returns a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive.

SortedMap<K,V>	**tailMap**(K fromKey)	Returns a view of the portion of this map whose keys are greater than or equal to fromKey.
NavigableMap<K,V>	**tailMap**(K fromKey, boolean inclusive)	Returns a view of the portion of this map whose keys are greater than (or equal to, if inclusive is true) fromKey.

Methods declared in interface Map

clear, compute, computeIfAbsent, computeIfPresent, containsKey, containsValue, equals, forEach, get, getOrDefault, hashCode, isEmpty, merge, put, putAll, putIfAbsent, remove, remove, replace, replace, replaceAll, size

Methods declared in interface SequencedMap

sequencedEntrySet, sequencedKeySet, sequencedValues

Methods declared in interface SortedMap

comparator, entrySet, firstKey, keySet, lastKey, putFirst, putLast, values

Method Details

lowerEntry

Map.Entry<K,V> lowerEntry(K key)

Returns a key-value mapping associated with the greatest key strictly less than the given key, or null if there is no such key.

Parameters:

key - the key

Returns:

an entry with the greatest key less than key, or null if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

lowerKey

`K lowerKey(K key)`

Returns the greatest key strictly less than the given key, or `null` if there is no such key.

Parameters:

`key` - the key

Returns:

the greatest key less than `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

floorEntry

`Map.Entry<K,V> floorEntry(K key)`

Returns a key-value mapping associated with the greatest key less than or equal to the given key, or `null` if there is no such key.

Parameters:

`key` - the key

Returns:

an entry with the greatest key less than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

floorKey

`K floorKey(K key)`

Returns the greatest key less than or equal to the given key, or `null` if there is no such key.

Parameters:

`key` - the key

Returns:

the greatest key less than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

ceilingEntry

```
Map.Entry<K,V> ceilingEntry(K key)
```

Returns a key-value mapping associated with the least key greater than or equal to the given key, or `null` if there is no such key.

Parameters:

`key` - the key

Returns:

an entry with the least key greater than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

ceilingKey

```
K ceilingKey(K key)
```

Returns the least key greater than or equal to the given key, or `null` if there is no such key.

Parameters:

`key` - the key

Returns:

the least key greater than or equal to `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

higherEntry

`Map.Entry<K,V> higherEntry(K key)`

Returns a key-value mapping associated with the least key strictly greater than the given key, or `null` if there is no such key.

Parameters:

`key` - the key

Returns:

an entry with the least key greater than `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

higherKey

`K higherKey(K key)`

Returns the least key strictly greater than the given key, or `null` if there is no such key.

Parameters:

`key` - the key

Returns:

the least key greater than `key`, or `null` if there is no such key

Throws:

`ClassCastException` - if the specified key cannot be compared with the keys currently in the map

`NullPointerException` - if the specified key is null and this map does not permit null keys

firstEntry

`Map.Entry<K,V> firstEntry()`

Returns a key-value mapping associated with the least key in this map, or `null` if the map is empty.

Specified by:

`firstEntry` in interface `SequencedMap<K,V>`

Returns:

an entry with the least key, or `null` if this map is empty

lastEntry

`Map.Entry<K,V> lastEntry()`

Returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty.

Specified by:

`lastEntry` in interface `SequencedMap<K,V>`

Returns:

an entry with the greatest key, or `null` if this map is empty

pollFirstEntry

`Map.Entry<K,V> pollFirstEntry()`

Removes and returns a key-value mapping associated with the least key in this map, or `null` if the map is empty (optional operation).

Specified by:

`pollFirstEntry` in interface `SequencedMap<K,V>`

Returns:

the removed first entry of this map, or `null` if this map is empty

Throws:

`UnsupportedOperationException` - if the `pollFirstEntry` operation is not supported by this map

pollLastEntry

`Map.Entry<K,V> pollLastEntry()`

Removes and returns a key-value mapping associated with the greatest key in this map, or `null` if the map is empty (optional operation).

Specified by:

`pollLastEntry` in interface `SequencedMap<K,V>`

Returns:

the removed last entry of this map, or `null` if this map is empty

Throws:

`UnsupportedOperationException` - if the `pollLastEntry` operation is not supported by this map

descendingMap

`NavigableMap<K,V> descendingMap()`

Returns a reverse order view of the mappings contained in this map. The descending map is backed by this map, so changes to the map are reflected in the descending map, and vice-versa. If either map is modified while an iteration over a collection view of either map is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined.

The returned map has an ordering equivalent to `Collections.reverseOrder(comparator())`. The expression `m.descendingMap().descendingMap()` returns a view of `m` essentially equivalent to `m`.

Returns:

a reverse order view of this map

navigableKeySet

`NavigableSet<K> navigableKeySet()`

Returns a `NavigableSet` view of the keys contained in this map. The set's iterator returns the keys in ascending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

Returns:

a navigable set view of the keys in this map

descendingKeySet

`NavigableSet<K> descendingKeySet()`

Returns a reverse order `NavigableSet` view of the keys contained in this map. The set's iterator returns the keys in descending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through

the iterator's own `remove` operation), the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

Returns:

a reverse order navigable set view of the keys in this map

subMap

```
NavigableMap<K,V> subMap(K fromKey,
                         boolean fromInclusive,
                         K toKey,
                         boolean toInclusive)
```

Returns a view of the portion of this map whose keys range from `fromKey` to `toKey`. If `fromKey` and `toKey` are equal, the returned map is empty unless `fromInclusive` and `toInclusive` are both true. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside of its range, or to construct a submap either of whose endpoints lie outside its range.

Parameters:

`fromKey` - low endpoint of the keys in the returned map

`fromInclusive` - `true` if the low endpoint is to be included in the returned view

`toKey` - high endpoint of the keys in the returned map

`toInclusive` - `true` if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this map whose keys range from `fromKey` to `toKey`

Throws:

`ClassCastException` - if `fromKey` and `toKey` cannot be compared to one another using this map's comparator (or, if the map has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if `fromKey` or `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` or `toKey` is null and this map does not permit null keys

IllegalArgumentException - if fromKey is greater than toKey; or if this map itself has a restricted range, and fromKey or toKey lies outside the bounds of the range

headMap

```
NavigableMap<K,V> headMap(K toKey,
                          boolean inclusive)
```

Returns a view of the portion of this map whose keys are less than (or equal to, if inclusive is true) toKey. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an IllegalArgumentException on an attempt to insert a key outside its range.

Parameters:

toKey - high endpoint of the keys in the returned map

inclusive - true if the high endpoint is to be included in the returned view

Returns:

a view of the portion of this map whose keys are less than (or equal to, if inclusive is true) toKey

Throws:

ClassCastException - if toKey is not compatible with this map's comparator (or, if the map has no comparator, if toKey does not implement Comparable). Implementations may, but are not required to, throw this exception if toKey cannot be compared to keys currently in the map.

NullPointerException - if toKey is null and this map does not permit null keys

IllegalArgumentException - if this map itself has a restricted range, and toKey lies outside the bounds of the range

tailMap

```
NavigableMap<K,V> tailMap(K fromKey,
                          boolean inclusive)
```

Returns a view of the portion of this map whose keys are greater than (or equal to, if inclusive is true) fromKey. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Parameters:

`fromKey` - low endpoint of the keys in the returned map

`inclusive` - `true` if the low endpoint is to be included in the returned view

Returns:

a
view of the portion of this map whose keys are greater than (or equal to, if `inclusive` is true) `fromKey`

Throws:

`ClassCastException` - if `fromKey` is not compatible with this map's comparator (or, if the map has no comparator, if `fromKey` does not implement `Comparable`). Implementations may, but are not required to, throw this exception if `fromKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` is null and this map does not permit null keys

`IllegalArgumentException` - if this map itself has a restricted range, and `fromKey` lies outside the bounds of the range

subMap

```
SortedMap<K,V> subMap(K fromKey,
                      K toKey)
```

Returns a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive. (If `fromKey` and `toKey` are equal, the returned map is empty.) The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Equivalent to `subMap(fromKey, true, toKey, false)`.

Specified by:

`subMap` in interface `SortedMap<K,V>`

Parameters:

`fromKey` - low endpoint (inclusive) of the keys in the returned map

`toKey` - high endpoint (exclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive

Throws:

`ClassCastException` - if `fromKey` and `toKey` cannot be compared to one another using this map's comparator (or, if the map has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if `fromKey` or `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `fromKey` or `toKey` is null and this map does not permit null keys

`IllegalArgumentException` - if `fromKey` is greater than `toKey`; or if this map itself has a restricted range, and `fromKey` or `toKey` lies outside the bounds of the range

headMap

`SortedMap<K,V> headMap(K toKey)`

Returns a view of the portion of this map whose keys are strictly less than `toKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Equivalent to `headMap(toKey, false)`.

Specified by:

`headMap` in interface `SortedMap<K,V>`

Parameters:

`toKey` - high endpoint (exclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys are strictly less than `toKey`

Throws:

`ClassCastException` - if `toKey` is not compatible with this map's comparator (or, if the map has no comparator, if `toKey` does not implement `Comparable`). Implementations may, but are not required to, throw this exception if `toKey` cannot be compared to keys currently in the map.

`NullPointerException` - if `toKey` is null and this map does not permit null keys

IllegalArgumentException - if this map itself has a restricted range, and toKey lies outside the bounds of the range

tailMap

SortedMap<K,V> tailMap(K fromKey)

Returns a view of the portion of this map whose keys are greater than or equal to fromKey. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an IllegalArgumentException on an attempt to insert a key outside its range.

Equivalent to tailMap(fromKey, true).

Specified by:

tailMap in interface SortedMap<K,V>

Parameters:

fromKey - low endpoint (inclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys are greater than or equal to fromKey

Throws:

ClassCastException - if fromKey is not compatible with this map's comparator (or, if the map has no comparator, if fromKey does not implement Comparable). Implementations may, but are not required to, throw this exception if fromKey cannot be compared to keys currently in the map.

NullPointerException - if fromKey is null and this map does not permit null keys

IllegalArgumentException - if this map itself has a restricted range, and fromKey lies outside the bounds of the range

reversed

default NavigableMap<K,V> reversed()

Returns a reverse-ordered view of this map. The encounter order of mappings in the returned view is the inverse of the encounter order of mappings in this map. The reverse ordering affects all order-sensitive operations, including those on the view collections of the returned view. If the implementation permits modifications to this view, the modifications "write through" to the

underlying map. Changes to the underlying map might or might not be visible in this reversed view, depending upon the implementation.

This method is equivalent to `descendingMap`.

Specified by:

`reversed` in interface `SequencedMap<K,V>`

Specified by:

`reversed` in interface `SortedMap<K,V>`

Implementation Requirements:

The implementation in this interface returns the result of calling the `descendingMap` method.

Returns:

a reverse-ordered view of this map, as a `NavigableMap`

Since:

21

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.